

Relatório de Code Smells — Projeto CliniDesk

Este documento apresenta os principais *code smells* identificados no projeto CliniDesk, considerando o back-end da aplicação. Os problemas encontrados envolvem principalmente segurança, robustez e organização arquitetural.

1. Mass Assignment

Tipo do smell:

Mass Assignment / Unsafe Data Exposure

Arquivo:

`patientController.js`

Código identificado:

```
const novoAgendamento = new Agendamento(req.body);
```

Problema encontrado

O controller utilizava diretamente o `req.body` para criar um novo agendamento. Isso significa que qualquer campo enviado pelo cliente era automaticamente aceito pela aplicação.

Refatoração realizada

Foi feita uma desestruturação explícita dos campos permitidos antes de criar o objeto, garantindo que apenas os dados esperados sejam processados pela aplicação.

2. Falta de validação de entrada

Tipo do smell:

Missing Validation

Arquivo:

`patientController.js`

Código identificado:

```
res.status(400).json({ error: error.message });
```

Problema encontrado

Os dados recebidos pela API não passavam por uma validação adequada antes de serem enviados ao banco de dados.

Refatoração realizada

Foi criado o middleware `validateAgendamento`, responsável por validar:

- telefone;
- data;
- horário;
- setor;
- campos obrigatórios.

Com isso, os dados são verificados antes de chegar ao controller, tornando a API mais segura e organizada

3. Rate limiting no login

Smell:

Missing Security Protection

Problema

O endpoint de login aceitava infinitas tentativas.

Refatoração

Adição de `express-rate-limit`:

```
const rateLimit = require('express-rate-limit');
```

```
const loginLimiter = rateLimit({
```

```
  windowMs: 15 * 60 * 1000,
```

```
  max: 5
```

```
});
```

4. Condicionais complexas e acoplamento estático no modal

- **Tipo do smell:** Nested Conditionals (Arrow Anti-Pattern) / Hardcoded Data
- **Arquivo:** `ModalNovoAgendamento.jsx` (Front-end)
- **Código identificado:** Bloco de validação aninhado (`if` dentro de `if`) e renderização manual de strings na tag `<select>`.
- **Problema encontrado:** O componente continha estruturas de validação profundamente aninhadas que dificultavam a leitura e manutenção (*Arrow Anti-Pattern*). Além disso, utilizava dados de clínicas fixados diretamente no JSX (*hardcoded*), limitando a flexibilidade do sistema.
- **Refatoração realizada:** * Aplicação de **Guard Clauses (Early Return)** nas validações do formulário, barrando e interrompendo a execução imediatamente ao encontrar o primeiro campo obrigatório inválido.
 - Substituição das opções estáticas do `<select>` por um mapeamento dinâmico (`.map()`) da propriedade `clinicas` recebida via *props*.
 - Correção do estado de inputs utilizando curto-circuito (`valores.paciente || ''`) para assegurar consistência do ciclo de vida e evitar componentes não controlados no React.

5. Código duplicado, strings mágicas e complexidade ciclomática

- **Tipo do smell:** Duplicate Code / Magic Strings / Conceptual Bloat
- **Arquivo:** `Dashboard.jsx` (Front-end)
- **Código identificado:** Uso repetido da função `format()` do `date-fns`, repetição de textos puros ("`Geriatría`", "`Nutrição`") e múltiplos objetos de mapeamento visual na função de cores.
- **Problema encontrado:** O componente acumulava responsabilidades demais (*God Component*). A formatação de datas era replicada em múltiplos trechos (filtros e impressão de tickets), strings estruturais de setores eram escritas de forma manual correndo riscos de erros de digitação, e o gerenciamento visual de cores usava mapeamentos redundantes (`cores` e `textColors`).
- **Refatoração realizada:**
 - **Extract Method:** Extração da lógica de tratamento cronológico para a função utilitária `formatarDataBR()`, centralizando o reaproveitamento do layout `dd/MM/yyyy`.
 - **Replace Magic Literal:** Criação do dicionário estático imutável `const SETORES`, padronizando os identificadores de setores em um único ponto e removendo acoplamentos por strings literais.
 - **Simplify Conditional / Map Clean-up:** Simplificação drástica da função `getSetorColor()` por meio da unificação das propriedades visuais em um único dicionário chave-valor (`configuracaoCores`), diminuindo sensivelmente a complexidade ciclomática do arquivo.